

# module-01-python-basics

## Chapter 1: Python Basics | 30 programs in learning order

*Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). PPT Module-1: intro, expressions, datatypes, variables, print/input/len/str-int-float, operators*

### How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

### Concepts in detail

Everything is an expression that evaluates to one value. Values have types (int, float, str); operators combine them. input() ALWAYS returns a string, so convert with int()/float() before math, and with str() before joining to text. Your first real program (hello.py) is just: print, input, variables, len, and type conversion glued together.

### Key ideas to carry forward

- $2 + 2$  evaluates to 4; 'Alice' + 'Bob' evaluates to 'AliceBob' (concatenation).
- 'Alice' \* 5 repeats the string; 'Alice' + 42 is a TypeError (convert first).
- spam = 40 stores a value; spam = spam + 2 overwrites it.
- Variable names: one word, letters/numbers/underscore, never start with a number.
- print() shows output; input() reads a line as str; len() counts characters.

### Classic beginner mistakes

- ! Beginners mix up = (store) with == (compare) - that comes in Module 02.
- ! Forgetting int(input()) then wondering why spam \* 10 repeats text instead of doing math.
- ! Unclosed quotes give 'EOL while scanning string literal'.

*Study order hint: Read programs 01-12 to master the interactive shell, then 13+ to see the same ideas as saved programs.*

## Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

### Q1. What is the difference between an expression and a statement?

A: An expression evaluates to a single value ( $2 + 2$  becomes 4). A statement does something (`spam = 42` stores a value). You can put expressions inside statements, e.g. inside `print()`.

### Q2. What does `input()` return, and why does it matter?

A: Always a string, even if the user types digits. So `int(input())` before math, else `'3' + 4` raises `TypeError` and `'3' * 3` repeats text.

### Q3. Explain `+` and `*` on strings.

A: `+` joins two strings; `*` repeats a string by an integer. Any other mix (`'a' * 'b'`, `'a' * 2.5`) is a `TypeError`.

### Q4. `int` vs `float` vs `str` - give conversions.

A: `int(3.9)` truncates to 3; `float('3.1')` is 3.1; `str(29)` is '29' for joining into messages. `int('3.5')` fails - parse float first.

### Q5. What happens on `5 +` in the shell?

A: `SyntaxError`: Python cannot parse it. Syntax errors are caught before running; fix the grammar, often a missing value or quote.

### Q6. Why does Python use indentation?

A: Indentation defines code blocks instead of braces. Same indent = same block; it forces readable structure.

## Teach it to others

### The one analogy that sticks

A variable is a labeled jar on a shelf: `spam = 42` puts the number 42 in the jar labeled spam. Putting something new in replaces the old - jars hold one thing at a time.

### What your students will get wrong

`x input()` gives numbers automatically (it gives strings).

`x print()` quotes appear on screen (quotes only mark the string).

`x =` and `==` are interchangeable (store vs compare).

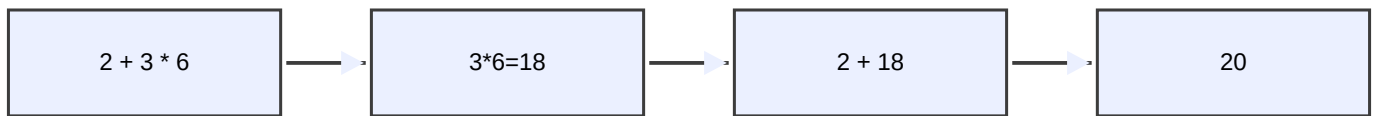
### 5-minute whiteboard plan

1. Ask: what is  $2 + 3 * 4$ ? Reveal precedence with live shell eval.
2. Demo `TypeError` with `'age: ' + 20`, then fix with `str(20)`.
3. Build `hello.py` together line by line; each student predicts output before F5.
4. Exit ticket: write `int/float/str` conversions for `'25'`.

## Visualize it - draw these from memory

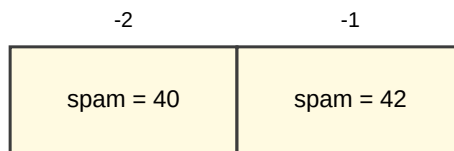
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

### Visual 1 - an expression evaluates down to one value



Read left to right: Python shrinks each part until one value remains. Draw this pipeline for any expression.

### Visual 2 - a variable is a labeled jar (one thing inside)



Second assignment overwrites: the jar holds 42 now, 40 is gone. Whiteboard: draw jar, erase, rewrite.

## More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

### X1. Age next year (types in action)

```
age = input('Age? ')
print('Next year: ' + str(int(age) + 1))
# EXPECTED: Age? 20 -> Next year: 21
```

### X2. Bill splitter

```
total = float(input('Total? '))
n = int(input('People? '))
print('Each pays: ' + str(total / n))
# EXPECTED: Total? 100, People? 4 -> Each pays: 25.0
```

### X3. Name badge

```
name = input('Name? ')
print(('Hi ' + name + '! ') * 3)
print('Letters: ' + str(len(name)))
# EXPECTED: Name? Al -> Hi Al! Hi Al! Hi Al! / Letters: 2
```

## Program map - all 30 at a glance

```
01 01_entering_expressions_interactive.py
02 02_entering_expressions_interactive_2.py
03 03_entering_expressions_interactive_3.py
04 04_entering_expressions_interactive_4.py
05 05_entering_expressions_interactive_5.py
06 06_integer_floating_point.py
07 07_string_concatenation_replication.py
08 08_string_concatenation_replication_alice.py
09 09_string_concatenation_replication_alice_2.py
10 10_string_concatenation_replication_alice_3.py
11 11_assignment_statements.py
12 12_assignment_statements_spam.py
13 13_your_first_program.py
14 14_your_first_program_2.py
15 15_comments.py
16 16_print_function.py
17 17_input_function.py
18 18_printing_user_s.py
19 19_len_function.py
20 20_len_function_2.py
21 21_len_function_3.py
22 22_len_function_4.py
23 23_str_int_float.py
24 24_str_int_float_2.py
25 25_str_int_float_spam.py
26 26_str_int_float_spam_2.py
27 27_str_int_float_spam_3.py
28 28_str_int_float_3.py
29 29_str_int_float_4.py
30 30_str_int_float_what.py
```

## Topic: Entering Expressions into the Interactive Shell

### 01/30 01\_entering\_expressions\_interactive.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

2 + 2

# OUT: 4

## 02/30 02\_entering\_expressions\_interactive\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
# OUT: Python 3.3.2 (v3.3.2:d047928ae3f6, May 16 2013, 00:06:53) [MSC v.1600 64 bit  
# OUT: (AMD64)] on win32  
# OUT: Type "copyright", "credits" or "license()" for more information.  
2 + 2  
# OUT: 4
```

### 03/30 03\_entering\_expressions\_interactive\_3.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
2
```

```
# OUT: 2
```



## 04/30 04\_entering\_expressions\_interactive\_4.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
2 + 3 * 6  
# OUT: 20  
(2 + 3) * 6  
# OUT: 30  
48565878 * 578453  
# OUT: 28093077826734  
2 ** 8  
# OUT: 256  
23 / 7  
# OUT: 3.2857142857142856  
23 // 7  
# OUT: 3  
23 % 7  
# OUT: 2  
2      +      2  
# OUT: 4  
(5 - 1) * ((7 + 1) / (3 - 1))  
# OUT: 16.0
```

## 05/30 05\_entering\_expressions\_interactive\_5.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

5 +
# OUT:   File "<stdin>", line 1
# OUT:   5 +
# OUT:   ^
# OUT: SyntaxError: invalid syntax
42 + 5 + * 2
# OUT:   File "<stdin>", line 1
# OUT:   42 + 5 + * 2
# OUT:   ^
# OUT: SyntaxError: invalid syntax
```

## Topic: The Integer, Floating-Point, and String Data Types

### 06/30 06\_integer\_floating\_point.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
'Hello world!
```

```
# OUT: SyntaxError: EOL while scanning string literal
```

## Topic: String Concatenation and Replication

### 07/30 07\_string\_concatenation\_replication.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
'Alice' + 'Bob'
```

```
# OUT: 'AliceBob'
```

## 08/30 08\_string\_concatenation\_replication\_alice.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'Alice' + 42  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#26>", line 1, in <module>  
# OUT:     'Alice' + 42  
# OUT: TypeError: Can't convert 'int' object to str implicitly
```

## 09/30 09\_string\_concatenation\_replication\_alice\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
'Alice' * 5
```

```
# OUT: 'AliceAliceAliceAliceAlice'
```

## 10/30 10\_string\_concatenation\_replication\_alice\_3.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

'Alice' * 'Bob'
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#32>", line 1, in <module>
# OUT:     'Alice' * 'Bob'
# OUT: TypeError: can't multiply sequence by non-int of type 'str'
'Alice' * 5.0
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#33>", line 1, in <module>
# OUT:     'Alice' * 5.0
# OUT: TypeError: can't multiply sequence by non-int of type 'float'
```

## Topic: Assignment Statements

### 11/30 11\_assignment\_statements.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 40
spam
# OUT:    40
eggs = 2
spam + eggs
# OUT:    42
spam + eggs + spam
# OUT:    82
spam = spam + 2
spam
# OUT:    42
```



## 12/30 12\_assignment\_statements\_spam.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = 'Hello'  
spam  
# OUT: 'Hello'  
spam = 'Goodbye'  
spam  
# OUT: 'Goodbye'
```

## Topic: Your First Program

### 13/30 13\_your\_first\_program.py [script]

*Script: save and run; predict the output first, then compare.*

```
# This program says hello and asks for my name.

print('Hello world!')
    print('What is your name?')    # ask for their name
myName = input()
print('It is good to meet you, ' + myName)
print('The length of your name is:')
    print(len(myName))
print('What is your age?')    # ask for their age
    myAge = input()
    print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```

## 14/30 14\_your\_first\_program\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

# OUT: Python 3.3.2 (v3.3.2:d047928ae3f6, May 16 2013, 00:06:53) [MSC v.1600 64 bit
# OUT: (AMD64)] on win32
# OUT: Type "copyright", "credits" or "license()" for more information.
===== RESTART =====

# OUT: Hello world!
# OUT: What is your name?
# OUT: Al
# OUT: It is good to meet you, Al
# OUT: The length of your name is:
# OUT: 2
# OUT: What is your age?
# OUT: 4
# OUT: You will be 5 in a year.
```

## Topic: Comments

### 15/30 15\_comments.py [script]

*Script: save and run; predict the output first, then compare.*

```
# This program says hello and asks for my name.
```

## Topic: The print() Function

### 16/30 16\_print\_function.py [script]

*Script: save and run; predict the output first, then compare.*

```
print('Hello world!')  
    print('What is your name?') # ask for their name
```

## Topic: The input() Function

**17/30 17\_input\_function.py [script]**

*Script: save and run; predict the output first, then compare.*

```
myName = input()
```

## Topic: Printing the User's Name

### 18/30 18\_printing\_user\_s.py [script]

*Script: save and run; predict the output first, then compare.*

```
print('It is good to meet you, ' + myName)
```

## Topic: The len() Function

### 19/30 19\_len\_function.py [script]

*Script: save and run; predict the output first, then compare.*

```
print('The length of your name is:')  
    print(len(myName))
```



## 20/30 20\_len\_function\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
len('hello')  
# OUT: 5  
len('My very energetic monster just scarfed nachos.')  
# OUT: 46  
len('')  
# OUT: 0
```

## 21/30 21\_len\_function\_3.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
print('I am ' + 29 + ' years old.')  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#6>", line 1, in <module>  
# OUT:     print('I am ' + 29 + ' years old.')  
# OUT: TypeError: Can't convert 'int' object to str implicitly
```

## 22/30 22\_len\_function\_4.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'I am ' + 29 + ' years old.'  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#7>", line 1, in <module>  
# OUT:     'I am ' + 29 + ' years old.'  
# OUT: TypeError: Can't convert 'int' object to str implicitly
```

## Topic: The str(), int(), and float() Functions

### 23/30 23\_str\_int\_float.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
str(29)
```

```
# OUT: '29'
```

```
print('I am ' + str(29) + ' years old.')
```

```
# OUT: I am 29 years old.
```

## 24/30 24\_str\_int\_float\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
str(0)
# OUT: '0'
str(-3.14)
# OUT: '-3.14'
int('42')
# OUT: 42
int('-99')
# OUT: -99
int(1.25)
# OUT: 1
int(1.99)
# OUT: 1
float('3.14')
# OUT: 3.14
float(10)
# OUT: 10.0
```

## 25/30 25\_str\_int\_float\_spam.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
spam = input()
```

```
# OUT: 101
```

```
spam
```

```
# OUT: '101'
```

## 26/30 26\_str\_int\_float\_spam\_2.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
spam = int(spam)
```

```
spam
```

```
# OUT: 101
```

## 27/30 27\_str\_int\_float\_spam\_3.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
spam * 10 / 5
```

```
# OUT: 202.0
```



## 28/30 28\_str\_int\_float\_3.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
int('99.99')
```

```
# OUT: Traceback (most recent call last):
```

```
# OUT:   File "<pyshell#18>", line 1, in <module>
```

```
# OUT:     int('99.99')
```

```
# OUT: ValueError: invalid literal for int() with base 10: '99.99'
```

```
int('twelve')
```

```
# OUT: Traceback (most recent call last):
```

```
# OUT:   File "<pyshell#19>", line 1, in <module>
```

```
# OUT:     int('twelve')
```

```
# OUT: ValueError: invalid literal for int() with base 10: 'twelve'
```

## 29/30 29\_str\_int\_float\_4.py [shell]

*Interactive: type each plain line into IDLE; # OUT: lines are expected results.*

# Interactive session from the book. Run line-by-line in IDLE.

# Lines starting with >>> are input; other lines are expected output (commented).

```
int(7.7)
```

```
# OUT: 7
```

```
int(7.7) + 1
```

```
# OUT: 8
```

### 30/30 30\_str\_int\_float\_what.py [script]

*Script: save and run; predict the output first, then compare.*

```
print('What is your age?') # ask for their age
myAge = input()
print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```